Separation of Concerns

`.h`, `.cpp` and `makefiles`

Department of Computer Science
University of Pretoria

2 April 2024

We are going to consider an example where we want to calculate the perimeter and area of Squares, Rectangles, Circles and Ellipses.

We will consider this example, beginning with the functions defined before the main function, using forward declarations and finally splitting the functions into .h and .cpp files.

- Square - $P = 4 * length$
- Rectangle - $P = 2 * (length + breadth)$
- Circle - $P = 2 * \pi * radius$
- Ellipse - Ramanujan approximation

$h = ((a-b)*(a-b))/((a+b)*(a+b))$

$P = \pi * (a + b) * (1 + ((3 * h)/(10 + \sqrt{(4 - 3 * h)})))$

where a is the major axis and b the minor axis

- Square - $A = length * length$
- Rectangle - $A = length * breadth$
- Circle - $A = \pi * (radius * radius)$
- Ellipse - $A = \pi * (a * b)$
  where a is the major axis and b the minor axis

Write a program using the following structure:

```
#include <iostream>
#include <cmath> // to use sqrt

const float PI = 3.14159265358979238;

using namespace std;

// definition and implementation of perimeter functions

// definition and implementation of area functions

int main() {
    // calls to the functions
    return 0;
}
```

Consider the following:

- Naming of the functions
- Ordering of the functions
- Can some of the functions be written in terms of others? If so, when do which functions needs to be defined? What does this do to the ordering of the functions?

Update the structure to the following:

```
#include <iostream>
#include <cmath> // to use sqrt

const float PI = 3.14159265358979323 8;

using namespace std;

// definition of perimeter functions
//  in the order square, rectangle, circle, ellipse

// definition of area functions
//  in the order square, rectangle, circle, ellipse

int main() {
    // calls to the functions
    return 0;
}

// implementation of perimeter functions

// implementation of area functions
```

- Is the code easier to read?
- Do the function implementations need to be in the same order as the function definitions?
- Do forward declarations solve some of the ordering issues?

- create a `Perimeter.h` and `Perimeter.cpp` file
- create an `Area.h` and an `Area.cpp` file
- place the function definitions in their appropriate files
- place the function implementation in their appropriate files

- update the file containing the main function to include the .h files
- where should PI be defined? It is not needed in the file with main.
- what about the inclusion of `cmath`?
- try and compile and link the .cpp files, What happens?

We require preprocessor directive to ensure that header (`.h`) files are not include multiple times. The process to do this is the following:

1. Ask the compiler to check if a macro has been defined
2. If it has not been defined, define the macro
   - include the necessary definitions
3. mark the end of the definitions (macro)

1. Ask the compiler to check if a macro has been defined -
   `#ifndef MacroName_h`
2. Define the macro -
   `#define MacroName_h`
   - include the necessary definitions
3. mark the end of the macro -
   `endif /* MacroName_h */`

For example `Perimeter.h` will be defined as follows:

```
#ifndef Perimeter_h
#define Perimeter_h

        // Perimeter function declarations
        float peri_square(float);
        float peri_rectangle(float, float);
        float peri_circle(float);
        float peri_ellipse(float, float);

#endif /* Perimeter_h */
```

The corresponding implementation file (`Perimeter.cpp`):

```
#include <cmath>

#include "Perimeter.h"
#include "Constants.h"

// Perimeter functions implementations
```

What is defined in `Constants.h`?

An example main:

```
#include <iostream>

#include "Perimeter.h"
#include "Area.h"

using namespace std;

int main() {
        cout << peri_square(10) << endl;
        cout << peri_rectangle(10,20) << endl;
        cout << peri_circle(10) << endl;
        cout << peri_ellipse(10,20) << endl;
        cout << endl;
        cout << area_square(10) << endl;
        cout << area_rectangle(10,20) << endl;
        cout << area_circle(10) << endl;
        cout << area_ellipse(10,20) << endl;
        return 0;
}
```

You now have 6 files that need to be managed.

- The `Perimeter.h` and `Area.h` files act as forward declarations for the functions (i.e. function definitions)
- `Constants.h` provides the definition of PI and is included in the `Perimeter.cpp` and `Area.cpp` files
- `Perimeter.cpp` and `Area.cpp` provide the function implementations
- The `.cpp` with the main function (say `Functions.cpp`)

- **Compile** the .cpp files
- **Link** the .o files, either by using the command-line command: `g++ Perimeter.o Area.o Functions.o -o Functions` or `g++ Area.o Perimeter.o Functions.o -o Functions`
- **Execute** the program: `./Functions`

Compile and link using a program called `make`.

Makefile Version 1 - Automate Compiling and Linking
(create a file called `Makefile` containing the following instructions)

```
Functions: Perimeter.o Area.o Functions_V3.o
        g++ Perimeter.o Area.o Functions_V3.o -o Functions

Perimeter.o:  Perimeter.cpp Perimeter.h Constants.h
        g++ -c Perimeter.cpp

Area.o:  Area.cpp Area.h Constants.h
        g++ -c Area.cpp

Functions.o:  Functions.cpp Perimeter.h Area.h
        g++ -c Functions.cpp
```

Next lecture....

- Look at examples of functions with different definitions.
- Consider makefiles that use macros and rules